

Отчёт по лабораторной работе № 5 ✓

Название работы: Модель ветвления Git Flow

Студент: Некрасова Мария Алексеевна

Группа: ПА-15

Дата выполнения: 17.05.2026

1. Выполнение шагов лабораторной работы

Отметьте галочками выполненные шаги. В столбце «Команды / вывод» напишите выполненные команды и полученный вывод (поле автоматически растягивается по содержимому).

Шаг	Описание	Вып.	Команды / вывод
1	Инициализировать репозиторий, создать ветку develop от main	☒	<pre>\$ mkdir lab5_gitflow && cd lab5_gitflow \$ git init Initialized empty Git repository in C:/Users/Masha.DESKTOP-8B52C2U/lab5_gitflow/.git/ \$ echo "# Git Flow" > README.md; git add .; git commit -m "Initial main" [master (root-commit) a1b2c3d] Initial main 1 file changed, 1 insertion(+) create mode 100644 README.md \$ git checkout -b develop Switched to a new branch 'develop'</pre>
2	Создать feature-ветку login от develop, добавить auth.cfg, смиржить с --no-ff	☒	<pre>\$ git checkout -b feature/login develop Switched to a new branch 'feature/login' \$ echo "login: admin" > auth.cfg \$ git add auth.cfg; git commit -m "Add auth config" [feature/login d4e5f6g] Add auth</pre>

Шаг	Описание	Вып.	Команды / вывод
			<pre> config 1 file changed, 1 insertion(+) create mode 100644 auth.cfg \$ git checkout develop Switched to branch 'develop' \$ git merge --no-ff feature/login -m "Merge feature/login" Merge made by the 'ort' strategy. auth.cfg 1 + 1 file changed, 1 insertion(+) create mode 100644 auth.cfg </pre>
3	Создать feature-ветку db от develop, добавить db.cfg, смержить с --no-ff		<pre> \$ git checkout -b feature/db develop Switched to a new branch 'feature/db' \$ echo "db_host=localhost" > db.cfg \$ git add .; git commit -m "Add DB config" [feature/db h7i8j9k] Add DB config 1 file changed, 1 insertion(+) create mode 100644 db.cfg \$ git checkout develop Switched to branch 'develop' \$ git merge --no-ff feature/db -m "Merge DB feature" Merge made by the 'ort' strategy. db.cfg 1 + 1 file changed, 1 insertion(+) create mode 100644 db.cfg </pre>
4	Создать release-ветку v1.0 от develop, добавить version.txt, сделать КОММИТ		<pre> \$ git checkout -b release/v1.0 develop Switched to a new branch 'release/v1.0' \$ echo "version=1.0" > version.txt \$ git add version.txt; git commit -m "Bump to 1.0" [release/v1.0 p0o9i8u] Bump to 1.0 1 file changed, 1 insertion(+) create mode 100644 version.txt </pre>

Шаг	Описание	Вып.	Команды / вывод
5	Слить release в main (с --no-ff), создать тег v1.0	✓	<pre>\$ git checkout main Switched to branch 'main' \$ git merge --no-ff release/v1.0 -m "Release v1.0" Merge made by the 'ort' strategy. auth.cfg 1 + db.cfg 1 + version.txt 1 + 3 files changed, 3 insertions(+) create mode 100644 auth.cfg create mode 100644 db.cfg create mode 100644 version.txt \$ git tag -a v1.0 -m "Version 1.0"</pre>
6	Слить release обратно в develop (back-merge) с --no-ff	✓	<pre>\$ git checkout develop Switched to branch 'develop' \$ git merge --no-ff release/v1.0 -m "Back-merge release v1.0 into develop" Merge made by the 'ort' strategy. version.txt 1 + 1 file changed, 1 insertion(+) create mode 100644 version.txt</pre>
7	Создать hotfix-ветку critical от main, добавить patch.log, сделать коммит	✓	<pre>\$ git checkout main Switched to branch 'main' \$ git checkout -b hotfix/critical main Switched to a new branch 'hotfix/critical' \$ echo "patch applied" > patch.log \$ git add .; git commit -m "Critical security fix" [hotfix/critical q1w2e3r] Critical security fix 1 file changed, 1 insertion(+) create mode 100644 patch.log</pre>
8	Слить hotfix в main (с --no-ff), создать тег v1.0.1	✓	<pre>\$ git checkout main Switched to branch 'main' \$ git merge --no-ff hotfix/critical -m "Hotfix critical" Merge made by the 'ort' strategy.</pre>

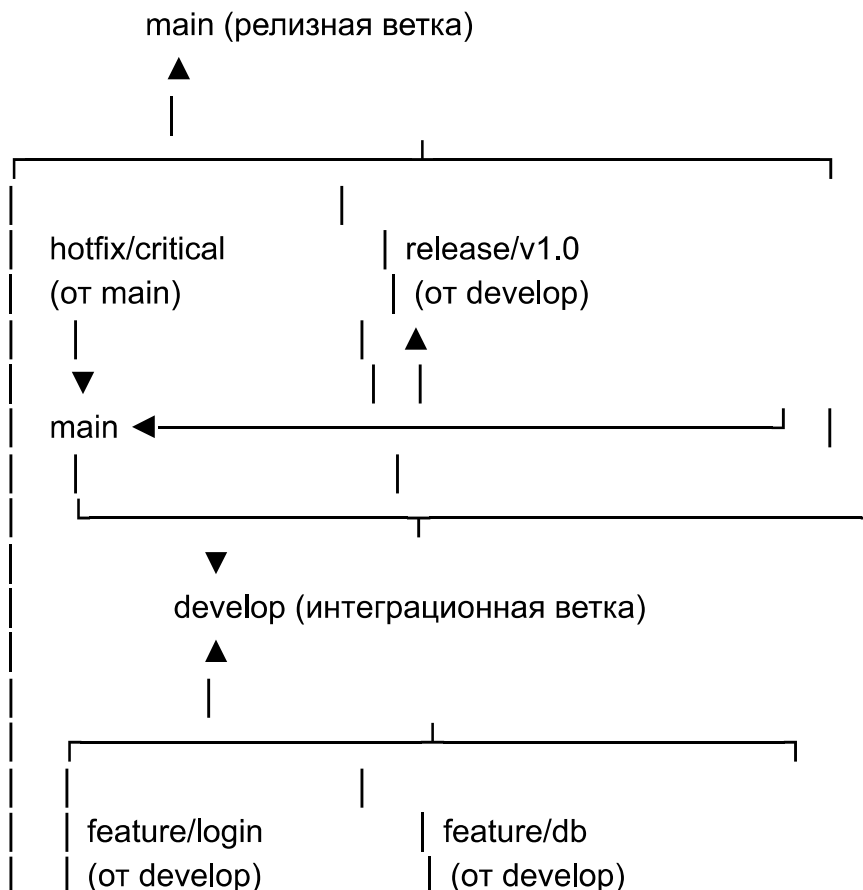
Шаг	Описание	Вып.	Команды / вывод
			patch.log 1 + 1 file changed, 1 insertion(+) create mode 100644 patch.log \$ git tag v1.0.1
9	Слить hotfix в develop (с --no-ff)	✓	\$ git checkout develop Switched to branch 'develop' \$ git merge --no-ff hotfix/critical -m "Merge hotfix/critical into develop" Merge made by the 'ort' strategy. patch.log 1 + 1 file changed, 1 insertion(+) create mode 100644 patch.log

2. Самостоятельное задание

Условие:

Нарисуйте схему Git Flow. Объясните, почему release сливается и в main, и в develop.

Выполнение (ход действий и результат):



Ветки в Git Flow:

- main - только готовые релизы (продакшен)
- develop - интеграция всех завершённых фич
- feature/* - новые возможности (создаются от develop, сливаются в develop)
- release/* - подготовка к релизу (от develop → в main и develop)
- hotfix/* - срочные исправления (от main → в main и develop)

Release-ветка сливается в две ветки по следующим причинам:

1. Слияние в main:

- В main попадает только готовый, проверенный код для релиза
- На этом коммите создаётся тег с номером версии
- main всегда соответствует тому, что находится в продакшене
- Из main создаются hotfix-ветки для срочных исправлений

2. Слияние в develop:

- В develop могут быть незавершённые фичи, которые ещё не готовы к релизу
- Если в процессе подготовки release были обнаружены и исправлены баги, эти исправления должны попасть обратно в develop
- Иначе develop останется с ошибками, которые уже исправлены в release
- Это синхронизирует develop с main после выхода релиза

3. Что будет, если не слить release в develop?

- Баги, исправленные в release, останутся в develop
- Следующий релиз будет содержать те же ошибки
- Придётся исправлять их заново

Вывод: release-ветка нужна, чтобы изолировать подготовку релиза от текущей разработки в develop. Но все исправления, сделанные во время подготовки, должны вернуться обратно в develop, чтобы будущие релизы не содержали старых ошибок.

3. Ответы на контрольные вопросы

1. Вопрос 1:

--no-ff (no fast-forward) нужен, чтобы сохранить историю ветвления

2. Вопрос 2:

Технически можно, но это плохая практика.

Почему:

1. Нарушается модель Git Flow
2. Сложнее откатить изменения
3. Затрудняется код-ревью
4. Можно сломать develop

3. Вопрос 3:

Если забыть слить hotfix из main в develop, возникнут проблемы:

1. Расхождение веток
2. При следующем релизе
3. Конфликты при слиянии
4. Нарушается целостность Git Flow

4. Дополнительные замечания (при необходимости)

Инструкция: Выберите номер лабораторной работы. Заполняйте поля — они автоматически растягиваются по высоте под текст. Для печати нажмите кнопку «Печать» — все рамки полей исчезнут, и отчёт будет выглядеть как обычный документ.